

WEEK 8

Inheritance and Runtime Polymorphism

Solutions and short notes

Topics: Inheritance • Method Overriding • Runtime Polymorphism • is-a Relationship • Dynamic Method Dispatch • Polymorphic Collections • Code Reusability & Extensibility

1. Key Concepts

Inheritance

Inheritance lets a child class reuse fields and methods from a parent class using **extends**. Use it when the child is genuinely a kind of the parent.

Method Overriding

A subclass provides its own implementation of an inherited instance method with the same method name and compatible signature.

Runtime Polymorphism

A parent-class reference can refer to a child-class object. When an overridden method is called, the object's actual class determines the implementation.

is-a Relationship

An is-a relationship means one type is a specialized form of another. For example, *Car is a Vehicle*. A has-a relationship describes containment, not inheritance.

Dynamic Method Dispatch

Java selects the overridden method at runtime based on the actual object, not just the reference variable's declared type.

Polymorphic Collections

An array or collection of a parent type can store objects of different child types, allowing the same method to be called on each.

Code Reusability & Extensibility

Shared code stays in the parent class. New child classes can add or override behavior without rewriting the code that processes the parent type.

2. Java Example: Inheritance, Overriding and Dynamic Dispatch

This short program demonstrates all the main ideas in one place.

```
class Vehicle {
    void startEngine() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Car starts with a key");
    }
}

class Motorcycle extends Vehicle {
    @Override
    void startEngine() {
        System.out.println("Motorcycle starts");
    }
}

public class Main {
    public static void main(String[] args) {
        Vehicle[] vehicles = {
            new Car(),
            new Motorcycle(),
        }
    }
}
```

```

        new Vehicle()
    };

    for (Vehicle v : vehicles) {
        v.startEngine();
    }
}

```

Output

```

Car starts with a key
Motorcycle starts
Vehicle starts

```

Why it works: Car and Motorcycle inherit from Vehicle (is-a relationship). Both override startEngine(). The Vehicle array is a polymorphic collection. Each call uses dynamic method dispatch to run the method belonging to the actual object.

3. Answers to the Week 8 Quiz Questions

Q1. Which principle removes repeated if-else checks for document types?

Answer: C — Polymorphism. Call render() through a common Document reference; each object supplies its own implementation.

Q2. Which are genuine is-a relationships?

Answers: A, B and C. Car is a Vehicle; Rectangle is a Shape; DatabaseConnection is a NetworkResource. "HelperUtility contains a Calculator" is has-a.

Q3. What ensures the correct startEngine() method runs?

Answer: C — Runtime method dispatch. The actual object type selects the overridden method.

Q4. Shapes in a list call their own calculateArea() methods. What is this?

Answer: C — Inheritance-based polymorphism. The common Shape reference can refer to Circle and Rectangle objects.

Q5. Which statements about LibraryItem, Book and DVD are correct?

Answers: A, C and D. Book can extend inherited behavior; LibraryItem defines common behavior; a DVD can be handled through a LibraryItem reference. Option B is not necessarily correct because DVD's fixed loan period is specialized behavior.

Q6. Which statements about Animal, Dog and Cat are correct?

Answers: A and C. An Animal reference to a Dog calls Dog's overridden makeSound(); Dog's method is overridden behavior.

Q7. What is the benefit of adding WalletPayment to a polymorphic Payment collection?

Answer: C. WalletPayment can be added without changing the existing iteration logic in PaymentProcessor.

Q8. Why is inheritance for superficial reuse between unrelated classes inappropriate?

Answer: A. It creates tight coupling and a false is-a relationship, making the design rigid.

Q9. Advantages of polymorphism in a notification system?

Answers: A and C. A generic sender handles different notification types, and new channels can be added without changing sender logic.

Q10. Benefits of polymorphic collections?

Answers: A, B and C. They simplify iteration, allow uniform processing with specialized behavior, and reduce explicit type casting.

4. Answers to the Concept Questions

Q1. How does inheritance enable reuse and extension?

A base class stores common fields and methods, and subclasses inherit them. A subclass may add new methods or override an inherited method. Example: `Employee` contains `employeeId` and `name`; `SalariedEmployee` adds a monthly salary calculation, while `HourlyEmployee` calculates pay from hours and hourly rate.

Q2. Explain the is-a relationship.

An is-a relationship means the subclass is a specialized kind of its superclass, such as `Car` is a `Vehicle`. It matters because a child should be usable wherever the parent is expected. If this is not true, the design becomes confusing and fragile.

Q3. Define method overriding with an employee example.

Method overriding occurs when a subclass defines an instance method with the same signature as a method inherited from its parent. For example, `Employee` may define `calculateSalary()`; `SalariedEmployee` returns a fixed monthly salary, while `HourlyEmployee` returns `hoursWorked × hourlyRate`.

Q4. Explain runtime polymorphism / dynamic dispatch.

A variable declared as a base type can point to a subclass object. When an overridden instance method is called, Java checks the actual object's class at runtime and executes that class's implementation. For example, `Vehicle v = new Car(); v.startEngine()` calls `Car`'s `startEngine()`.

Q5. How do polymorphic collections work?

A collection declared with a base type can hold objects of different subclasses, such as `Vehicle[]` containing `Car` and `Motorcycle` objects. A loop can call `startEngine()` on every element, while each object performs its own behavior. This makes processing simpler and extensible.

Q6. Polymorphism versus repeated type conditions.

Repeated if-else or switch statements require central code to know every concrete type and must be updated when types are added. Polymorphism places each behavior in its class and lets common code call one shared method, reducing conditionals and improving maintainability.

Q7. How can a new PaymentMethod be added with minimal changes?

Define a common `Payment` base class or interface with a method such as `pay(amount)`. `CardPayment` and `BankTransferPayment` implement it, and a new `WalletPayment` implements the same operation. Code that processes `Payment` references can keep the same loop and call `pay()` on each object.

Q8. Inherited behavior versus overridden behavior.

Inherited behavior is a parent method used by a child without replacing it, suitable when the common behavior is already correct. Overridden behavior is a child-specific implementation, used when the child needs different behavior. Example: all vehicles may inherit `getBrand()`; `Car` and `Motorcycle` may override `startEngine()`.

Q9. Why not inherit only for code reuse?

Inheritance expresses an is-a relationship, not merely a way to copy code. Using it between unrelated classes causes tight coupling, exposes inappropriate methods, and makes future changes risky. Prefer composition or a shared helper when one object simply uses another.

Q10. VehicleRental with CarRental and TruckRental.

A `Rental` base class can hold common data and behavior such as `rentalId`, `duration`, `customer` details, and `calculateCost()`. `CarRental` may calculate a daily rate and mileage charge; `TruckRental` may include a capacity-based charge. Both can be processed as `Rental` objects, while overridden methods supply the specialized cost.

5. Quick Revision

- **extends:** creates a subclass relationship.
- **@Override:** marks a method intended to override an inherited method.
- **Base reference, child object:** `Vehicle v = new Car();`
- **Runtime dispatch:** the actual object's overridden instance method runs.
- **Polymorphic collection:** one base-type array/list can hold different subclass objects.
- **Good inheritance:** use it for a genuine is-a relationship.
- **Extensibility:** new subclasses can often be added without changing common processing code.